Readme.md file for Pipeline – By D.Sarath Sai

Description (for all 3 questions)

The code that I wrote performs all these tasks:

1. Loads the **Delhi-NCR** and **Delhi-Airshed** boundaries from GeoJSON files.
2. Loads the **WorldCover 2021 land-cover raster**.
3. Ensures that all layers share the same **Coordinate Reference System (CRS)**.
4. Clips the raster to the **Delhi-NCR** region.
5. Plots the results, showing the land-cover classes from the raster, overlaid with **red boundaries** for Delhi-NCR and **blue boundaries** for the airshed.

The resulting map provides a visual overview of the study area **before any image filtering or further analysis**.

Importing Libraries

Before working with raster and spatial data, we import **rioxarray**. To understand its role, here's a comparison with related libraries:

- **NumPy** – A library for fast numerical computations with arrays of numbers.
- **Xarray** – Builds on NumPy by adding **labels and names** to arrays, making it easier to handle multi-dimensional scientific data.
- **rioxarray** – Extends Xarray with **geospatial tools** for working with raster datasets, including CRS handling, spatial clipping, and raster I/O.

Example:

1. Numpy :

```python
import numpy as np
data = np.array([[1, 2], [3, 4]])

# access element at row 0, column 1
print(data[0, 1])  # Output: 2

# to access entire row 1
print(data[1])     # Output: [3 4]
```

```
2
[3 4]
```

2. Xarray

```python
import xarray as xr
xarr = xr.DataArray([[1, 2], [3, 4]], dims=["x", "y"], coords={"x": [10, 20], "y": [100, 200]})

# Access element at x=10, y=200, sel meaning select subsets
print(xarr.sel(x=10, y=200).item())  # Output: 2

# Access entire row with x=20, sel meaning select subsets
print(xarr.sel(x=20))
```

```
2
<xarray.DataArray (y: 2)> Size: 16B
array([3, 4])
Coordinates:
  * y        (y) int64 16B 100 200
    x        int64 8B 20
```

3. RioXArray – not built-in, so import it with "!pip install rioxarray" – in colab

```
import rioxarray
xarr.rio.write_crs("EPSG:4326", inplace=True)

# Access element by coordinates (x=10, y=200)
print(xarr.sel(x=10, y=200).item())

# Check CRS (geospatial info)
print(xarr.rio.crs)
print("by sarath")
```

```
2
EPSG:4326
by sarath
```

And other main Packages are:

```
import geopandas as gpd #handles geospatial vector data
import rasterio #reads and writes raster (grid) geospatial data
import shapely #performs geometric operations on shapes
import matplotlib.pyplot as plt #for plots(for some others, we use seaborn)
import geemap #Interactive mapping and analysis with Google Earth Engine
import torch #Deep learning framework for tensor computations
import torchvision #Computer vision tools and pretrained models for PyTorch
import seaborn
import sklearn #we need to install scikit-learn
```

The dataset contains two types of files:

- **PNG files** – representing the image data
- **GeoJSON files** – representing the geographical data structures

For handling the GeoJSON files, we use the Python libraries **GeoPandas**, **Rasterio**, and **Rioxarray** for spatial data processing and raster operations.

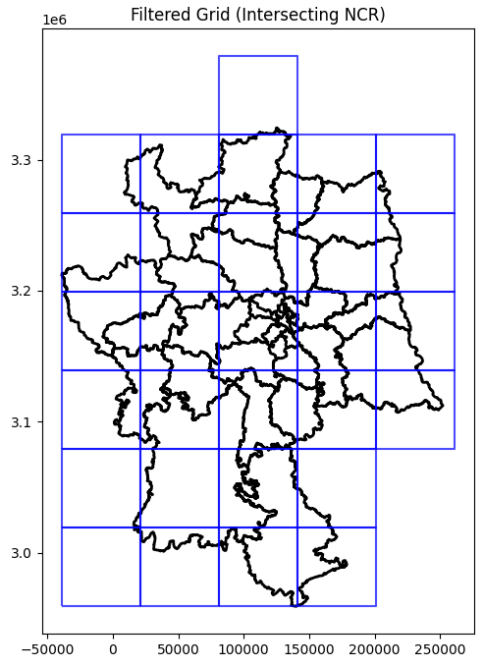
What does **CRS mean**:- A **Coordinate Reference System (CRS)** defines how spatial data (local, regional, or global) is represented on a flat map or a globe using coordinate values such as latitude and longitude.

EPSG:4326 is a widely used geographic CRS based on the **WGS84** datum. It represents the Earth as a three-dimensional ellipsoid and uses latitude and longitude in decimal degrees. EPSG codes are standard identifiers used to specify the CRS of geographic data, including points, lines, and polygons.

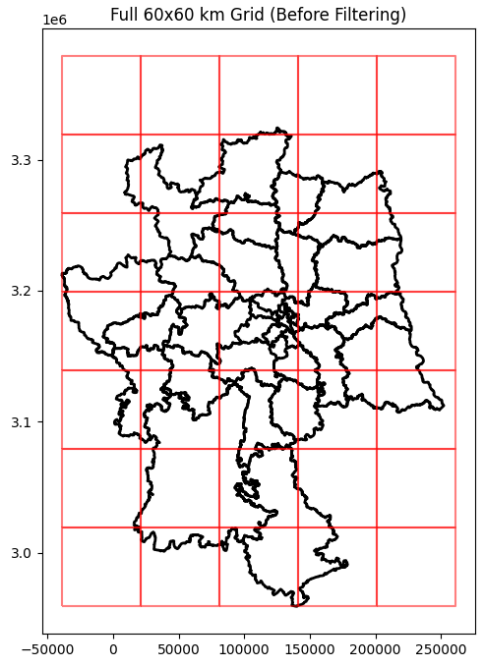
A **Raster Coordinate Reference System (CRS)** defines how the pixels in a raster dataset (such as a satellite image or elevation model) correspond to real-world geographic locations. It ensures that each pixel is correctly mapped to its actual position on the Earth's surface.

The spatial extent of Delhi NCR is approximately **51.9 km in length** and **48.48 km in width** (according to Wikipedia). For simplicity, we initially considered a **60 km × 60 km grid**.

After aligning the grid with the actual study area, we removed the unnecessary grid squares outside the region of interest. As a result, the effective coverage was reduced from **60 × 60 km** to approximately **52 × 48.5 km**, corresponding to the actual dimensions of Delhi NCR.



From



We filtered the images based on the defined grid. Initially, there were **9,216 total images**, which were reduced to **8,015 images** after filtering. The removed **1,201 images** were located outside the NCR region.

We have a total of **8,015 images**, which are divided into two parts:

- **Training set:** 60% of the data
- **Test set:** 40% of the data

The ESA land cover classes are assigned as follows:

- **10, 20, 30 – Vegetation** (Tree cover, Shrubland, Grassland)
- **40 – Cropland**
- **50 – Built-up**
- **60 – Bare land** (completely empty land)
- **70 – Snow/Ice**
- **80 – Water**
- **90 – Wetland**
- **95 – Mangroves** (specialized trees and shrubs that grow in tropical and subtropical coastal intertidal zones)
- **100 – Moss/Lichen** (Lichens are not mosses or plants; they are composite organisms formed through a symbiotic relationship between fungi and algae or cyanobacteria.)

In the context of Delhi:

- Classes **60 (Bare land)** and **70 (Snow/Ice)** are generally not present.
- Class **95 (Mangroves)** is also not found in this region.
- Class **100 (Moss/Lichen)** may exist, but only in very small areas and not in large concentrations.

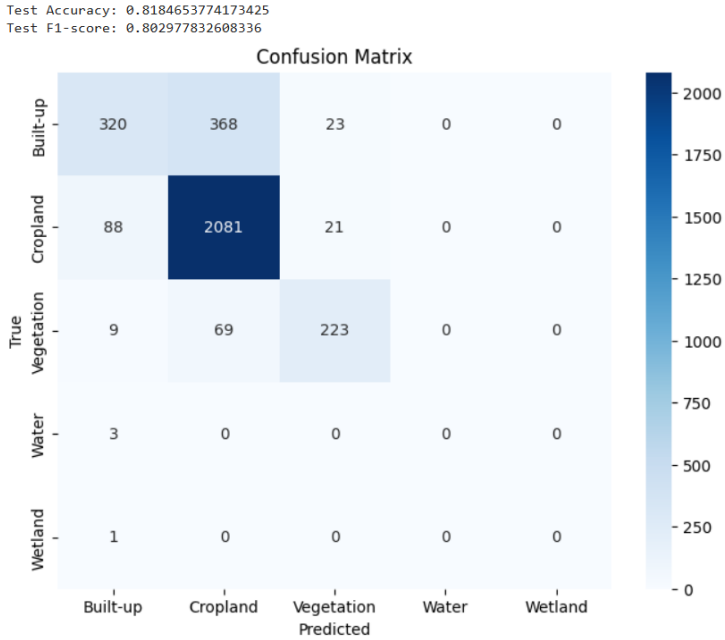
```
Total images after cleaning: 8015
Train size: 4809
Test size: 3206

Class distribution:
land_use
Cropland      5474
Built-up      1778
Vegetation     754
Water            7
Wetland          2
Name: count, dtype: int64
```

Image locations are matched to the raster, small square areas around them are extracted, and the most common land-cover type in each area is identified, ignoring any points outside the raster.

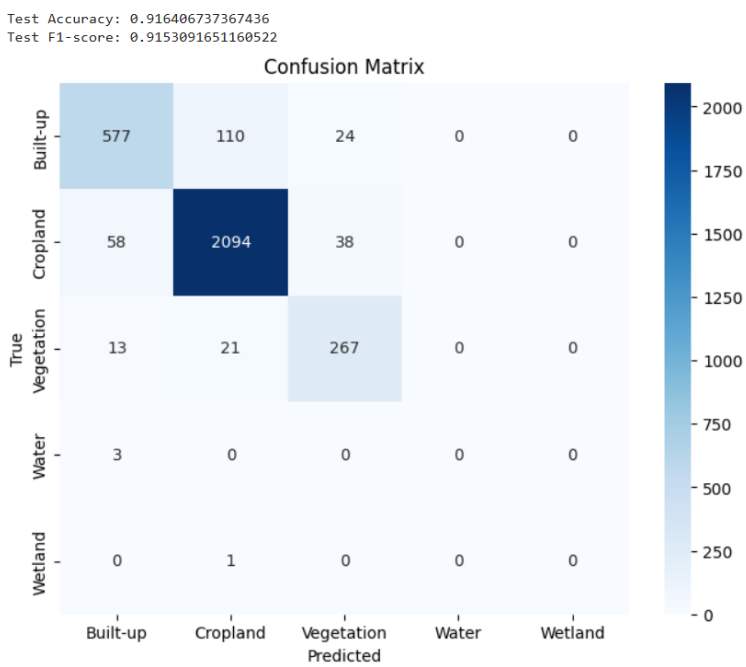
Models used:-

And Now, I have trained the models – Tiny CNN's and ResNet

- **Tiny CNN(Basic CNN that I have created)** - This model is a small convolutional neural network with **3 convolutional layers** (16, 32, and 64 filters respectively), each followed by ReLU activation and max-pooling. After feature extraction, the network uses **2 fully connected (dense) layers**, with **128 neurons in the hidden layer** and an output layer equal to the number of land-use classes. The input patches are 128×128 images converted to 3 channels. The model was trained for 5 epochs using Adam optimizer and cross-entropy loss. It achieved a **test accuracy of 81%**, indicating good performance for a lightweight CNN architecture.



- **ResNet(Residual Network)** – This model is a pretrained deep convolutional neural network with **18 layers** (17 convolutional layers plus 1 fully connected layer). The final fully connected layer is replaced to match the number of land-use classes, with neurons equal to len(classes). ResNet-18 contains an initial convolution layer, followed by **8 residual blocks**, and ends with a dense classification layer. Transfer learning is applied by fine-tuning the pretrained weights using Adam optimizer for 5 epochs. This deeper architecture typically performs better than a small CNN due to its residual connections and larger representational capacity.



Accuracy: 91.64%, F1 Score – 91.530%

Important Note: Increasing CNN complexity increases representational capacity and often training time, but it does not guarantee perfect performance due to overfitting, optimization challenges, and data limitations.